ENTSCHEIDUNG UND AUSGANGSLAGE

Der Weg zum ersten Pilot

Empfehlung: mit vorhandener Hardware starten, einen überschaubaren Pilot betreiben und den späteren Umzug in ein Rechenzentrum von Anfang an vorbereiten.

Die beiden Rechner stehen voraussichtlich bei Jakob zu Hause. Du möchtest sie von deinem Zuhause aus verwalten und CampusZeit öffentlich vorführen sowie mit einer ersten Hochschule erproben. Die virtuellen Umgebungen sollen getrennte Aufgaben haben und kontrolliert miteinander kommunizieren. Dieser Plan beschreibt Einrichtung und Reihenfolge; er installiert noch nichts und enthält keine Buchung oder Kontaktaufnahme.

Rechner	Bekannter Stand	Vorgesehene Aufgabe
Dell Precision 3440	Intel Core i5, 10. Generation; RAM und Speicher erweitert	Proxmox und getrennte VMs für Anwendung, Datenbank und Tests
HP Pro Mini	Intel Core i3; Speicher erweitert	Eigener Sicherungsrechner; später Wiederherstellung prüfen

Was bereits im Projekt vorhanden ist

React/TypeScript-Oberfläche, Django-Backend, PostgreSQL-Konfiguration, Redis, Celery-Planungsworker, Dockerfile und Compose-Datei. Es gibt eine Betriebsanleitung, Kundenanlage und einen lokalen Backupdienst. Die neue iPhone-App und automatische Ausfallmitteilungen liegen noch auf dem offenen App-Branch; sie sind vor einer Pilotbereitstellung als geprüfte Version zusammenzuführen. [R1, R2]

Die Reihenfolge

1. Hardware und Jakobs Internetanschluss prüfen; vorhandene Rechnerdaten sichern und ersten Vor-Ort-Termin vereinbaren.

2. Privaten Betriebsaufbau einrichten und Wiederherstellung testen.

3. Domain, HTTPS und eine öffentliche Demo mit fiktiven Daten bereitstellen.

4. Pilotgespräche führen; erst nach Freigabe echte Hochschuldaten übernehmen.

Offen: tatsächliche RAM-Größen, Laufwerke und freie Kapazitäten, genauer i3, Jakobs Router/Internetanbieter, öffentliche IP oder DS-Lite, Uploadrate und Zugriff auf den Router. Hardware wurde hier nicht vermessen. Die unten genannten Größen sind Startannahmen, kein Kapazitätsnachweis.

ZIELARCHITEKTUR FÜR EINEN PILOT

So kommunizieren die Umgebungen



Beide Rechner stehen bei Jakob. Website/Landingpage bleiben separat, etwa auf Vercel. Der Pilot nutzt für Oberfläche und API dieselbe Hochschuladresse. Gateway und VPN sind geplant, noch nicht eingerichtet.

Verbindung	Erlaubter Zweck	Begrenzung
Browser / iPhone > Gateway	HTTPS auf 443; 80 für Umleitung/Zertifikat	Nur vorgesehene Domain
Gateway > Anwendungs-VM	Webzugriff über Tailscale	Nur App-Port; kein Zugriff auf Verwaltung
Anwendungs-VM > Daten-VM	PostgreSQL 5432, Redis 6379	Nur App/Worker und passende Konten
Dein PC > Serververwaltung	SSH 22, Proxmox 8006, Backup 8007	Nur freigegebene Betreiber über Tailscale
Server > HP Mini / extern	Abgeschlossene Sicherungen	Eigene Sicherungskonten und Schlüssel

Proxmox trennt die VM-Umgebungen; Docker Compose steuert darin die Dienste. Firewalls begrenzen Verbindungen zwischen Anwendung, Daten und Administration. Datenbankports bleiben privat. [1, 3, 4]

VIRTUALISIERUNG UND GETRENNTE INSTANZEN

Die Rechner einrichten

1. Bestand aufnehmen, bevor etwas installiert wird

In BIOS/UEFI Virtualisierung prüfen. RAM, SSD/HDD-Modell, Gesundheitszustand, freien Speicher und bestehende Projekte erfassen. Vorhandene Daten extern sichern. Proxmox-Installation kann den gewählten Datenträger überschreiben: erst nach geprüftem Backup und bewusst gewähltem Ziellaufwerk installieren. Proxmox benötigt neben seinem eigenen Speicher zusätzliche Ressourcen für die VMs. [1]

Umgebung auf dem Precision	Startannahme	Inhalt
app-pilot	4 vCPU / 4 GB RAM / 60 GB	Django, Oberfläche, Celery, Medienvolume
data-pilot	2 vCPU / 4 GB RAM / 80 GB	PostgreSQL 17 und privates Redis
test-campuszeit	2 vCPU / 4 GB RAM / 60 GB	Getrennter Teststack mit eigener Datenbank
Proxmox / Reserve	Mindestens 4 GB RAM frei	Host, Dateisystem und Puffer

Für diese Zuordnung mindestens 16 GB nutzbaren RAM prüfen; 32 GB geben mehr Spielraum. vCPUs sind Zuteilungen, keine zusätzlichen physischen Kerne. Bei knappen Ressourcen die Test-VM nur bei Bedarf starten. Solver-CPU und RAM zuerst messen; Parallelität anfangs auf einen Planungsauftrag begrenzen.

2. Welche Betriebssysteme installieren?

Precision: aktuelle stabile Proxmox-VE-ISO direkt auf der Hardware. App-, Daten- und Test-VM: Ubuntu Server 24.04 LTS (amd64), ohne Desktop; diese Version erhält reguläre Sicherheitswartung bis Mai 2029. Gateway: ebenfalls Ubuntu Server 24.04 LTS. SSH-Schlüssel, Updates und Firewall einrichten. Interne Adressen mit Jakobs und deinem Heimnetz abgleichen. Docker läuft in den VMs, nicht direkt auf dem Proxmox-Host. [1, 14]

3. HP Mini als eigener Backuprechner

HP Mini: wenn RAM und Laufwerke passen, die aktuelle stabile Proxmox-Backup-Server-ISO direkt installieren, mit eigenem Systemlaufwerk und ausreichend Sicherungsspeicher. Als Planwert 8 GB RAM plus Speicherbedarf prüfen; der Hersteller empfiehlt mindestens 4 GiB plus zusätzlichen RAM pro TiB. Auf diesem Rechner keine produktive Datenbank unterbringen, von der die Anwendung ständig abhängt. [2]

4. Später eine Instanz je Hochschule

Für den ersten Pilot ein eigenes App-/Daten-Paar. Eine weitere Hochschule erhält eigene Laufzeit, Datenbank, Datenbankkonto, Redis, Medien, Geheimnisse, Domain und Sicherungen. Gleicher freigegebener Code, getrennte Konfiguration. Eine Subdomain oder ein Git-Branch allein trennt keine Kundendaten. Vor jeder neuen Instanz Ressourcenbudget und Zugriffstrennung prüfen; kein Zwei-Rechner-HA-Cluster zum Start.

TAILSCALE FÜR BETRIEB UND ENTWICKLUNG

Dein Fernzugriff von zu Hause

Zwei Zugänge: Du verwaltest die Rechner privat über Tailscale. Studierende öffnen die CampusZeit-Domain ohne VPN. Der öffentliche Gateway erhält ausschließlich Zugriff auf den Webdienst.

Gerät / Dienst	Dein Zugriff	Freigabe
Precision / Proxmox	Browser: https://pve-jakob:8006	Nur Betreibergruppe; Proxmox-Konto bleibt nötig
App-VM	SSH oder VS Code Remote SSH	Dein Konto mit SSH-Schlüssel; regulär kein root
HP Mini / Backup Server	Browser: https://backup-jakob:8007	Nur Betreibergruppe und Sicherungskonten
Gateway im Rechenzentrum	SSH über Tailscale	Betreiber dürfen verwalten; Gateway selbst nur zur App

Die Gerätenamen sind Beispiele für MagicDNS. Standardzertifikate von Proxmox/PBS sind selbstsigniert; vor täglicher Nutzung passende Zertifikate oder einen privaten HTTPS-Proxy einrichten. Tailscale ersetzt nicht die Anmeldung in Proxmox/PBS. [15]

Einmal vor Ort, danach von deinem PC aus

1. Bei Jakob die Betriebssysteme mit Bildschirm/USB-Stick installieren, Netzwerk anschließen und DHCP-Reservierungen setzen. Lokalen Zugang zuerst testen. Bei ausgefallenem Internet bleibt ein Ansprechpartner vor Ort erforderlich.

2. Ein eigenes CampusZeit-Tailnet anlegen. Du und Jakob erhalten getrennte Konten mit MFA. Tailscale auf deinem Windows-PC, dem Proxmox-Host, der App-VM, dem HP-Backuprechner und dem Gateway einrichten. Linux-Pakete aus der offiziellen Anleitung beziehen. [15, 17]

3. Server als getaggte Geräte registrieren; Betreibergruppe und konkrete Zugriffregeln festlegen. Gateway nur zu App-Port 8000; keine Regeln für Datenbank, Proxmox oder Backupverwaltung. Breite Standardfreigaben ersetzen und negative Zugriffstests durchführen. [16, 18]

4. Bei direkter Geräteaufnahme kein gesamtes Heimnetz und keinen Exit-Node freigeben. Wenn direkte Hostinstallation unerwünscht ist: eigene Management-VM als Subnet-Router, ausschließlich nötige Einzeladressen (/32), Routenfreigabe und Zugriffregeln. [19]

5. Von zu Hause/Mobilfunk Browser und SSH testen. Neustart prüfen: erst Tailscale, dann an dessen IP gebundener Webdienst; VMs automatisch starten. Schlüsselablauf, Gerätewiderruf und Alarmierung konfigurieren; Backupzugang testen.

Subnet-Router ist eine Alternative, kein zusätzlich benötigter Dienst im empfohlenen Direktaufbau. Proxmox-Konsolen helfen bei VM-Problemen; ein komplett ausgefallener Precision lässt sich darüber nicht reparieren. Jakob benötigt eine kurze Notfallanleitung.

FERNZUGRIFF ALS FESTE BETRIEBSANFORDERUNG

Server mit Codex verwalten

Die Infrastruktur wird so vorbereitet, dass Iman sie mit Hilfe von Codex vom freigegebenen Arbeitsrechner aus betreiben kann: Status prüfen, Fehler untersuchen, Instanzen anlegen, Updates einspielen und Sicherungen kontrollieren.

Codex nutzt den SSH-Client für deinen konkreten Auftrag. Voraussetzung: der Arbeits- oder verbundene Ausführungsrechner ist in der Sitzung erreichbar, Tailscale läuft und die nötigen Rechte sind verfügbar. Ein unabhängiger Dauerzugang oder unbeaufsichtigte Hintergrundüberwachung entsteht dadurch nicht.

Vorbereitung	Zweck
SSH-Aliasse: campuszeit-pve, -backup, -gateway, -pilot-a	Zielserver schnell und eindeutig wählen
Widerrufbares Betriebskonto und lokaler SSH-Schlüssel	Zugang außerhalb von Chat und GitHub verwalten
Server-/Instanzinventar ohne Passwörter	Rolle, Einrichtungskennung, Projekt, Version und Backupziel zuordnen
Geprüfte Hostschlüssel und nichtinteraktive Anmeldung	Automatisierte Befehle stoppen bei fehlender Freigabe

Einmal einrichten, danach gezielte Aufträge

1. Getrennte Benutzer und Tailscale-Regeln einrichten. Server mit Betriebskonto und lokal erzeugtem, geschütztem Schlüssel verbinden; nur öffentliche Schlüssel auf Servern hinterlegen.

2. Host-Fingerprints vertrauenswürdig prüfen und SSH-Aliasse konfigurieren. Für Befehle BatchMode und strikte Hostprüfung verwenden, keine Agentweitergabe. Benötigte Schlüssel lokal entsperren. [26]

3. Lesende Diagnosen und gezielte Wartungsrechte einrichten. Docker-Zugriff entspricht umfassender Hostmacht; eingeschränkte Rechte benötigen bewusst definierte Werkzeuge. [27]

4. Im selben Ausführungskontext Verbindung, Zielidentität, Version, Logs und Backupstatus prüfen. Nach einem Neustart erneut testen; bei Ausfall von Internet oder Hardware übernimmt Jakob die Arbeiten vor Ort.

Ein Auftrag wie "Prüfe pilot-a" betrifft genau die passende Instanz. Updates folgen Freigabeversion, Backup und Funktionsprüfung. Unklare Ziele oder Datenüberschreibungen werden geklärt; Geheimnisse und nicht benötigte Kundeninhalte bleiben geschützt.

Vorbereitet: separate Betriebsanleitung, SSH-Beispiel und Beispielinventar unter docs/operations. Der lokale SSH-Client ist vorhanden; die Alias-Syntax wurde ohne Verbindung geprüft. Tatsächliche Serveradressen, Konten, Schlüssel und Rechte fehlen noch. Keine Serververbindung wurde aufgebaut. [R4]

VOM VORHANDENEN PROJEKT ZUM BETRIEB

CampusZeit in den VMs betreiben

Die aktuelle Compose-Datei startet alle Dienste auf einem gemeinsamen Host. Für zwei VMs muss sie gezielt aufgeteilt werden; diese Konfiguration ist noch nicht umgesetzt.

Vorhanden	Für diesen Aufbau anzupassen
Dockerfile: regulärer Build, Gunicorn; Compose: web und worker	App/Worker auf app-pilot; Start nach Abnahme einer festen Codeversion
Compose-Datenbankname db; Redisname redis	Für VM-übergreifende Verbindungen private Adressen und DB-/Redis-Authentifizierung konfigurieren
Webport nur 127.0.0.1:8000	Gateway-Zugang ausschließlich auf der Tailscale-/privaten Adresse binden und per Firewall begrenzen
Medienvolume und backup.sh	Medien auf app-pilot sichern; Backupdienst auf neue DB-Adresse und externes Ziel abstimmen

1. Getrennte Betriebsdateien für App-VM und Daten-VM erstellen. Produktionsdaten nicht aus der lokalen SQLite-Testdatenbank ungeprüft übernehmen.

2. PostgreSQL und Redis privat starten. Datenbankrolle nur für diese Instanz; PostgreSQL-Zugriff auf App-IP begrenzen, TLS für TCP-Verbindungen vorsehen. Redis mit Authentifizierung und geschütztem Netz betreiben. [4]

3. Geheimnisse außerhalb von Git hinterlegen. DEBUG=0, LOCAL_WORKER=0, SECRET_KEY, POSTGRES_HOST und REDIS_URL setzen. Docker-Secrets benötigen beim jetzigen Django-Setup zusätzlich eine passende Datei-Leseintegration; _FILE wird nicht automatisch verarbeitet. [3]

4. Image bauen, Migrationen einmal vor Web/Worker-Start ausführen und ein gesondertes Betriebskonto anlegen. Keine seed_demo- oder seed_showcase-Daten im Kundenbetrieb.

5. Einrichtung über den vorhandenen provision-Befehl anlegen; Kundenkonto ohne staff-/Superuserrechte. Lizenzende und verantwortliche Person festhalten. [R1]

6. Django-Hosts und CSRF-Ursprünge auf die echte HTTPS-Domain setzen. Bei TRUST_PROXY=1 darf nur der vertrauenswürdige Proxy zugreifen; er überschreibt X-Forwarded-Proto. [R1]

7. Anmeldung, Planung/Worker, Veröffentlichung, öffentliche Anzeigen, QR-Zugang, Ausfallmitteilung und Backup-Wiederherstellung mit getrennten Testdaten prüfen.

Updateweg: freigegebener Commit > Test-VM > Abnahme > zusätzliches Backup > angekündigtes Wartungsfenster > Migration > neue Container > Funktionstest. Bei Datenbankmigrationen keine parallelen Schreibvorgänge; Rückkehr zur alten Version nur mit geprüftem Schema-/Restoreplan.

GLEICHE SOFTWARE - EIGENE LEERE DATEN

Neue Einrichtung aus einer Vorlage

Ziel: einmal den Betriebsaufbau vorbereiten. Danach startet ein Assistent pro Schule oder Hochschule die geprüfte Basis mit eigener Datenbank und eigenem Verwaltungszugang. Alle Funktionen sind vorhanden; fachliche Daten werden erst von der Einrichtung erfasst.

Drei Teile des Bereitstellungspakets

Teil	Einmal vorbereiten	Je Einrichtung
CampusZeit-Image	Freigegebene Version in GitHub Container Registry	Dasselbe Image für Webdienst und Worker; feste Versions-ID
Betriebsvorlage	Compose-Dateien für App-/Daten-VM plus Sicherungsregeln	Eigene Dienste, Datenbank, Redis, Medien und Backups
Einrichtungsassistent	Geführter Ablauf mit Prüfung, Protokoll und Wiederaufnahme	Name, Schulart, Domain, Verwaltungsadresse und Lizenzende

Ein Docker-Image enthält den Programmstand und seine Laufzeit. Kundendaten und Zugangsschlüssel liegen außerhalb in eigenen Datenträgern und geschützten Einstellungen. Eine neue Instanz nutzt frische leere Datenträger. Das Image wird aus einem geprüften Release gebaut und über seinen Digest festgelegt; ein wechselndes latest ist keine verbindliche Freigabe. [22]

So soll die Anlage für dich ablaufen

1. Neue Einrichtung starten: Name, Schule/Hochschule, Domain, Verwaltungs-E-Mail und Lizenzende eingeben. Der Assistent schlägt eine eindeutige Kennung vor und prüft freie Ressourcen sowie vorhandene Instanzen.

2. Vorbereiten: eigene App-/Daten-Umgebung, Geheimnisse und Sicherungsziele erzeugen; DNS, Gateway und Tailscale-Zugriffsregeln für genau diese Instanz prüfen.

3. Initialisieren: freigegebenes Image laden, leere Datenbank migrieren und die Einrichtung samt Verwaltungszugang über provision anlegen. Keine Demo- oder Kundendaten kopieren.

4. Prüfen: HTTPS, Anmeldung, Worker, Datenabgrenzung und Sicherung testen. Erst dann Betriebsadresse und Zugang sicher übergeben; Ergebnis und installierte Version dokumentieren.

Vorhanden: Dockerfile, Compose und provision. Am 13.10.2026 wurde provision nach Migration auf einer separaten frischen SQLite-Testdatenbank geprüft: eine Einrichtung, ein Verwaltungszugang ohne Superuserrechte und keine fachlichen Datensätze. Noch zu bauen: Release-Image-Pipeline, wiederverwendbare Produktionsdateien und Assistent. Docker-/Serverbetrieb ist durch diese Prüfung nicht nachgewiesen. [R1, R3]

ISOLATION, UPDATES UND PRÜFBARER ERFOLG

Instanzanlage verlässlich betreiben

Eine Schule erhält ihre eigene Umgebung

Passend zur geplanten VM-Trennung erhält jede Einrichtung ein eigenes App-/Daten-Paar, eigene Netzwerkfreigaben und eigene Einstellungen. Ein gemeinsames Image liefert den gleichen Funktionsumfang. Auf den jeweiligen Docker-Hosts bekommen die Stacks eindeutige Compose-Projektnamen. Keine fest gemeinsam benannten oder extern wiederverwendeten Kundendatenträger; Ports, Konten und Netzregeln separat festlegen. Projektnamen erleichtern Trennung, ersetzen aber keine Zugriffskontrolle. [23, 24]

Die Vorlage selbst bleibt neutral

Als zusätzliche Beschleunigung ist eine Ubuntu-VM-Vorlage mit Docker möglich. Sie enthält keine Kundenvolumes, persönlichen SSH-Schlüssel oder aktive Tailscale-Geräteidentitäten. Jeder Klon erhält eigene Maschinenidentität, SSH-Hostschlüssel und Tailscale-Registrierung. Bestehende Kunden-VMs oder deren Sicherungen werden nicht als Vorlage für einen neuen Kunden verwendet.

Fehler dürfen keine vorhandenen Daten ersetzen

1. Vor Änderungen Kennung, Zieladressen und Datenträgerzuordnung prüfen. Bei bereits vorhandener Einrichtung abbrechen oder nur einen dokumentierten unvollständigen Anlagenlauf fortsetzen.

2. Geheimnisse automatisch neu erzeugen, geschützt speichern und aus Logs/Git fernhalten. Ein Wiederholungsversuch setzt keine Datenbank zurück und ändert nicht still ein vorhandenes Passwort.

3. Zwischenschritte und Fehler klar protokollieren. Eine fehlgeschlagene Einrichtung bleibt als unvollständig erkennbar; kein Erfolg nur deshalb, weil ein Container läuft.

Updates werden je Instanz übernommen

Eine neue Version wird einmal gebaut, gegen eine leere Installation und ein Upgrade bestehender Testdaten geprüft und erst danach freigegeben. Jede Instanz behält zunächst ihre Version. Update mit Backup, Wartungsfenster, einmaligen Migrationen und Funktionsprüfung. Persistente Daten bleiben beim Austausch des Images bestehen. Ein alter Image-Digest alleine ist kein Rollback einer inkompatiblen Datenbankmigration. [22, 25]

Abnahme für das Paket

Zwei Einrichtungen aus derselben Vorlage aufbauen: beide fachlich leer, getrennte Konten und Datenträger; Anmeldung oder Datenzugriff auf die jeweils andere Einrichtung schlägt fehl. Zusätzlich doppelte Anlage, falsche Domain, fehlende Ressourcen, Unterbrechung/Wiederaufnahme, Neustart, Update mit erhaltenen Daten und Restore testen. Als Arbeitsziel sollen normale Neuanlagen nur die Eingaben und eine abschließende Prüfung benötigen; die Dauer wird erst nach einem echten Durchlauf zugesagt.

WEBSITE, PILOTADRESSE UND E-MAIL

Domain und öffentlicher Zugang

Erst Namen und Verfügbarkeit prüfen, dann eine Hauptdomain registrieren. Verlängerungskosten prüfen, Anbieterzugang mit MFA schützen und automatische Verlängerung aktivieren. Alle folgenden Adressen mit .example sind unverbindliche Beispiele; es wurde keine Domain reserviert.

Adresse als Beispiel	Ziel / Umsetzung
www.campuszeit.example	Jakobs Landingpage; eigenes Hostingprojekt und Landingpage-Branch
demo.campuszeit.example	Demo mit ausschließlich fiktiven Daten; getrennt vom Kundenbetrieb
pilot-a.campuszeit.example	Erste Hochschulinstanz: Oberfläche, /api und /wiki über Gateway
kontakt@campuszeit.example	Eigenes geschäftliches Postfach beim Mailanbieter

Empfohlener Zugang: kleiner Gateway in Deutschland

Ein kleiner Hetzner-VPS erhält die öffentliche Adresse. Gateway und App-VM werden als eigene, beschränkte Tailscale-Geräte verbunden. Caddy nimmt HTTPS an und reicht nur den CampusZeit-Webzugriff zur App-VM bei Jakob weiter. Studierende benötigen kein VPN. Die Verbindung wird mit Jakobs Anschluss getestet; eine eingehend erreichbare Heim-IP ist für diesen Entwurf nicht erforderlich. Proxmox und Backupverwaltung bleiben privat. [5, 6, 15, 16]

Alternative A: Cloudflare Tunnel für eine schnelle Demo, mit ausgehender Verbindung ohne offene Heimports. Vor echten Hochschuldaten zusätzlich Anbieter, Vertragslage und Datenfluss freigeben. Alternative B: öffentliche Heim-IP plus DNS-Aktualisierung und Portweiterleitung nur zum Proxy; zuerst Anschluss und Router prüfen. Keine Datenbank-, SSH- oder Proxmox-Freigabe am Heimrouter. [8]

Einrichtung in dieser Reihenfolge

1. Domain registrieren; einen verantwortlichen Inhaber und DNS-Anbieter festlegen.

2. Landingpage-Domain im Hostingprojekt hinzufügen. Die dort angezeigten A/CNAME/TXT-Werte exakt übernehmen; keine IP-Werte aus alten Beispielen kopieren. [9]

3. Pilot-Subdomain per A, gegebenenfalls AAAA, auf den Gateway legen. AAAA nur bei funktionierendem IPv6. Gateway, Tailscale und Caddy einrichten; Zertifikat und Umleitung auf HTTPS prüfen. [5, 10]

4. Geschäftliches Postfach anlegen; MX, SPF, DKIM und DMARC nach Mailanbieter einrichten. Versand und Antwort aus einem anderen Netzwerk testen.

Tailscale Funnel eignet sich als kurzfristiger Demozugang. Es verwendet ausschließlich .ts.net-Namen und feste Bandbreitenlimits; für die eigene CampusZeit-Domain ist der Gateway die Empfehlung. Proxmox/PBS niemals per Funnel veröffentlichen. Auf dem Gateway /admin und interne Betriebswege sperren; dafür privaten HTTPS-Zugang mit passenden Django-Hosts/CSRF-Ursprüngen einrichten. [21]

WAS DER MINI-PC LEISTET UND WAS OFFEN BLEIBT

Sichern, überwachen, wiederherstellen

Drei Kopien und ein Standort außerhalb des Hauses

Produktivdaten liegen auf dem Precision. Der HP Mini erhält unabhängige Sicherungen. Eine weitere verschlüsselte Kopie liegt außerhalb von Jakobs Haus, etwa bei dir oder in vertraglich passendem EU-Speicher. Beide Rechner bei Jakob teilen Strom-, Router- und Standortausfälle; der Mini-PC ist dort keine Sicherung an einem zweiten Standort.

Sicherung	Vorgeschlagener Start	Erfolgskriterium
Datenbank und Medien	Täglich und vor jedem Update; RPO-Ziel 24 Stunden	Passender DB-/Medienstand mit manifestierter Codeversion
VMs > HP Mini	Nächtlich; z.B. 7 tägliche / 4 wöchentliche / 3 monatliche Stände	Prüfung/Verifikation erfolgreich, Speicherbedarf gemessen
Verschlüsselte externe Kopie	Täglich; eigener Zugang und separate Schlüsselaufbewahrung	Nicht nur dieselbe Platte oder ungeschützter Dateisync
Wiederherstellung	Vor Pilotbeginn und dann monatlich	Leere Testumgebung startet; Anmeldung/Pläne/Dateien geprüft

Der heutige Backupdienst sichert täglich Datenbank und Medien nacheinander und bewahrt lokale Archive 14 Tage auf. Neue VM-Aufteilung, Ziele, Aufbewahrung und Backup-Überwachung müssen angepasst werden. Ein VM-Snapshot ersetzt kein Datenbank-/Dateibackup. Für konsistente DB- und Medienstände Schreibvorgänge im Sicherungsfenster pausieren oder einen abgestimmten Snapshot-Ablauf verwenden. [R1]

Ausfall erkennen, statt nur Dateien abzulegen

Von außerhalb des Heimnetzes HTTPS und /api/health/ überwachen. Zusätzlich Worker-Warteschlange, Testplanungsjob, letzte erfolgreiche Sicherung, freien Speicher und Zertifikat kontrollieren. Alarm an eine tatsächlich betreute Adresse senden. Monitoring nur auf dem HP Mini erkennt einen Ausfall des gesamten Hauses nicht zuverlässig.

Wartung und Ausfallziel

Als internes Pilotziel: höchstens 24 Stunden Datenverlust aus den täglichen Sicherungen und Wiederherstellung innerhalb eines Arbeitstags. Diese RPO-/RTO-Ziele sind erst nach einem echten Restoretest belastbar und noch kein SLA. UPS für Server, Router und Netzwerk prüfen. Der HP Mini ist kein automatisch umschaltender Ersatzserver; dortiges Wiederanlaufen nur nach gemessenem RAM-/Speicherbedarf planen.

Die Studierenden-App hält zuvor geladene Pläne und Mitteilungen offline vor. Neue Änderungen erreichen sie erst mit Verbindung. Ein Gateway hält die Adresse stabil, verhindert aber keinen Homeserver- oder Heimanschlussausfall. Vor verbindlichen Verfügbarkeitszusagen die produktive Instanz auf geeignetes Hosting umziehen. Mit Jakob Stromkosten, Zugang bei Störungen und Verantwortung für physische Arbeiten vereinbaren.

ABLAUF MIT ERGEBNISSEN STATT FESTEN VERSPRECHEN

Vom Aufbau zu Hochschulgesprächen

Phase ab Start	Arbeit	Ergebnis
Woche 1	Hardware/Netz prüfen, Domain wählen, Sicherungen und VM-Grundaufbau	Private Umgebung und gesicherter Bestand
Woche 2	Compose aufteilen, Testinstanz, HTTPS, Domain-Mail, Restoretest	Öffentliche Demo und dokumentierter Betriebsnachweis
Woche 2-3 parallel	5-10 passende Hochschulen recherchieren; warme Kontakte/Einführungen	2-3 Bedarfsgespräche als Arbeitsziel
Woche 3-4	Pilotumfang, Verantwortliche, Datenbedarf und Vertragsunterlagen klären	Ein vereinbarter, begrenzter Pilot
Danach 4-6 Wochen	Pilot begleiten, Fehler priorisieren, Aufwand/Last messen	Gemeinsame Entscheidung über nächsten Betriebsschritt

Zeitangaben sind grobe Planwerte, keine Lieferzusagen. Hardware, Rechte und Hochschulbeschaffung können länger dauern. Gespräche können bereits mit einer sauberen Demo beginnen; echte personenbezogene Daten folgen erst nach Freigabe.

Was wir vor dem ersten Gespräch bereitstellen

1. Eine Seite zum Nutzen: Stunden- und Prüfungsplanung, Raum-/Lehrendenbelegung, öffentliche Anzeigen, Studierenden-App. Zielgruppe und aktueller Funktionsumfang klar benennen.

2. Eine 15-minütige Demo mit erfundenen Personen, sichtbarem Veröffentlichungsablauf und Absagefall vorbereiten. Website mit Kontaktweg und vollständigen Betreiberangaben bereitstellen.

3. Für Studienorganisation/Stundenplanung und IT je einen passenden Kontakt ermitteln. Bestehende Kontakte um eine Einführung bitten oder vorgesehene Pilot-/Innovationswege nutzen.

4. Pilot begrenzen: ein Studiengang/Jahrgang, wenige planende Konten, feste Dauer und drei messbare Ziele, etwa Planerstellungszeit, Konflikte und Supportaufwand.

Vor Übernahme echter Daten

Mit der Hochschule Rollen, Auftragsverarbeitung, technische/organisatorische Maßnahmen, Unterauftragnehmer, Speicherorte, Lösch-/Exportweg und Notfallkontakt klären. Datenschutz- und IT-Verantwortliche müssen den konkreten Betrieb freigeben. Separate Instanzen ersetzen diese Prüfung nicht. Unaufgeforderte Werbe-E-Mails auch an Einrichtungen grundsätzlich nicht ohne passende Einwilligung senden; veröffentlichte Kontaktadressen alleine genügen nicht. [12, 13]

iPhone: derzeit lokale Testsignierung, keine reguläre Store-Verteilung; belastbaren Verteilungsweg für den Pilot festlegen. Sofortiger Push bei geschlossener App/APNs und Android bleiben offene Produktarbeiten. Im Gespräch nur geprüfte Eigenschaften zusagen. Lizenzmodell zunächst betreut je Einrichtung, mit Laufzeit, Leistungsumfang, Support und vereinbartem Ausstiegs-/Datenexportweg.

ENTSCHEIDUNGEN VOR DEM KAUF

Budget und erste Aufgaben

Posten	Grobe Planreserve	Annahme
Domain	10-30 EUR / Jahr	Endung und Verlängerung vor Kauf prüfen
Gateway-VPS	5-15 EUR / Monat	Kleine Instanz; IPv4/Steuern mitrechnen
Externes Backup	5-15 EUR / Monat	Abhängig von Kapazität und Aufbewahrung
E-Mail	0-10 EUR / Monat	Ein Geschäftspostfach; Anbieter prüfen
Strom bei Jakob	ca. 10-20 EUR / Monat	40-80 W gesamt, 720 h, 0,35 EUR/kWh

Grundbudget etwa 21-63 EUR monatlich einschließlich anteiliger Domain, ohne Tailscale-/Website-Tarif, Hardwarekauf, UPS, Arbeitszeit und Unterstützung. Tailscale Standard nennt derzeit 8 USD je Nutzer/Monat; für dich und Jakob also 16 USD zusätzlich, mögliche Steuern gesondert. Personal ist nur für nichtkommerzielle Nutzung vorgesehen. [20] Die EUR-Reserven sind eigene Budgetannahmen, keine Anbieterangebote. Vercel Hobby ist für nichtkommerzielle persönliche Nutzung. Für die Landingpage passenden Tarif einplanen; Vercel nennt derzeit 20 USD monatliche Pro-Plattformgebühr zuzüglich möglicher Nutzung, Zusatzplätze und Steuern. Preise vor Bestellung prüfen. [6, 11]

Die drei nächsten konkreten Aufgaben

1. Du und Jakob: RAM, Laufwerke und freie Kapazitäten erfassen, Jakobs Anschluss/Router prüfen, Daten sichern und Erstinstallation vor Ort planen. Dann entscheiden wir die endgültigen VM-Größen.

2. Gemeinsam: Hauptdomain und monatliches Budget einschließlich Tailscale-Tarif festlegen; dann Fernverwaltung und Gateway-Zugang abnehmen. Erst danach kaufen und DNS ändern.

3. Umsetzung: Betriebszugang für Iman/Codex, Release-Image, Produktionsvorlagen und Instanzassistent vorbereiten; leere Anlage, Update und Restore prüfen. Danach Demo und Pilotgespräche.

> Vor dem Datenpilot abhaken: HTTPS und Anmeldung extern getestet; keine offenen Datenports; Kunde ohne Superuserrechte; Geheimnisse geschützt; Wiederherstellung bestanden; Monitoring alarmiert; Hochschulfreigabe und Pilotumfang dokumentiert.

Ungeprüft: tatsächliche Hardwareleistung, Zugang durch den Heimanschluss, Domainverfügbarkeit, laufende Anbieterpreise und Anforderungen einer konkreten Hochschule. Keine Serverinstallation, Domainbestellung, DNS-Änderung oder E-Mail-Versendung wurde durch diesen Plan ausgeführt.

Arbeitsfassung vom 13.10.2026. Der folgende Quellenanhang macht Herstellerangaben, Projektstand und Planannahmen nachvollziehbar.

GEPRÜFT AM 13.10.2026

Quellen und Projektbezug

Hersteller- und Rechtsquellen belegen die jeweils markierten Aussagen. Auslegung, VM-Aufteilung, Zeitplan und Budget sind Vorschläge für CampusZeit. Diese Planung ersetzt keine konkrete Hardwaremessung oder vertragliche Betriebsfreigabe.

[1] Proxmox VE: Hardwareanforderungen
https://www.proxmox.com/en/products/proxmox-virtual-environment/requirements

[2] Proxmox Backup Server: Installation und Anforderungen
https://pbs.proxmox.com/docs/installation.html

[3] Docker: Compose-Secrets und Dateiübergabe
https://docs.docker.com/compose/how-tos/use-secrets/

[4] PostgreSQL: Hostzugriff und hostssl
https://www.postgresql.org/docs/current/auth-pg-hba-conf.html

[5] Caddy: Automatisches HTTPS
https://caddyserver.com/docs/automatic-https

[6] Hetzner Cloud: verfügbare Ressourcen und Preisprüfung
https://www.hetzner.com/cloud/

[7] WireGuard: Einrichtung und NAT-Keepalive
https://www.wireguard.com/quickstart/

[8] Cloudflare Tunnel: ausgehende Verbindungen
https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/

[9] Vercel: Eigene Domain hinzufügen
https://vercel.com/docs/domains/working-with-domains/add-a-domain

[10] DNS: A-, AAAA-, CNAME- und TXT-Einträge
https://developers.cloudflare.com/dns/manage-dns-records/reference/dns-record-types/

[11] Vercel: Hobby-Nutzungsgrenze / Pro-Kosten
https://vercel.com/docs/plans/hobby
https://vercel.com/docs/plans/pro-plan

[12] DSGVO: insbesondere Artikel 28 und 32
https://eur-lex.europa.eu/legal-content/EN-DE/TXT/?from=EN&uri=CELEX%3A32016R0679

[13] UWG § 7: elektronische Werbung und Einwilligung
https://www.gesetze-im-internet.de/uwg_2004/__7.html

[R1] Projekt: compose.yaml, Dockerfile, Backupskript und Betriebsanleitung
https://github.com/imannsv/campusTIME/blob/main/docs/OPERATIONS.md
https://github.com/imannsv/campusTIME/blob/main/compose.yaml

[R2] Aktuelle App-Entwicklung: PR #16, Stand 7bf29b5
https://github.com/imannsv/campusTIME/pull/16
https://github.com/imannsv/campusTIME/blob/codex/student-ios-app/docs/STUDENT_APP.md

Quellen zum Standort bei Jakob

[14] Ubuntu: Sicherheitswartung für 24.04 LTS
https://ubuntu.com/about/release-cycle

[15] Tailscale: direkte Integration auf Proxmox
https://tailscale.com/docs/integrations/proxmox

[16] Tailscale: Zugriffe beschränken
https://tailscale.com/docs/features/access-control

[17] Tailscale: Linux-Installation
https://tailscale.com/docs/install/linux

[18] Tailscale: Server-Tags und Schlüsselablauf
https://tailscale.com/docs/features/tags

[19] Tailscale: Subnet-Router, Routen und Regeln
https://tailscale.com/docs/features/subnet-routers

[20] Tailscale: kommerzielle Nutzung und Preise
https://tailscale.com/pricing

[21] Tailscale Funnel: Domains und Bandbreitenlimits
https://tailscale.com/docs/features/tailscale-funnel

Standortannahmen für die Umsetzung

Precision und HP Mini stehen voraussichtlich bei Jakob. Das Tailnet gehört zum CampusZeit-Betrieb; private Geräte und Netze werden nicht pauschal freigegeben. Sicherungen außerhalb von Jakobs Haus, physischer Zugang bei Störungen, Stromkosten und Routeränderungen werden vor dem Pilot gemeinsam vereinbart.

Ein öffentlicher Gateway gibt CampusZeit eine stabile HTTPS-Adresse. Er ersetzt keinen redundanten Heimanschluss und keinen Ersatzserver. Ausgehende Verbindung, direkte oder Relay-Verbindung und Uploadleistung werden am tatsächlichen Standort getestet.

Standortergänzung vom 13.10.2026. Keine Betriebssysteme, Tailscale-Konten, DNS-Einträge, Routerfreigaben oder VPS wurden durch die Dokumenterstellung eingerichtet.

RELEASE-IMAGE UND AUTOMATISIERTE BEREITSTELLUNG

Quellen zur wiederholbaren Anlage

[22] GitHub Container Registry: Images und feste Digests
https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

[23] Docker Compose: eindeutige Projektnamen
https://docs.docker.com/compose/how-tos/project-name/

[24] Docker Compose: Datenträger und Namenszuordnung
https://docs.docker.com/reference/compose-file/volumes/

[25] Docker Compose: Betrieb und Updates
https://docs.docker.com/compose/how-tos/production/

[R3] Vorhandene Kundenanlage: provision
https://github.com/imannsv/campusTIME/blob/main/backend/planner/management/commands/provision.py

[26] OpenSSH: Client- und Serverkonfiguration
https://man.openbsd.org/ssh_config
https://man.openbsd.org/sshd_config

[27] Docker Engine: Betriebsrechte und Sicherheit
https://docs.docker.com/engine/security/

[R4] CampusZeit: vorbereitete Betriebsanleitung und SSH-Beispiele
https://github.com/imannsv/campusTIME/tree/codex/infrastructure-plan/docs/operations

Geprüfte Grundlage und noch offene Umsetzung

Dockerfile und vorhandene Compose-Konfiguration wurden gelesen. Der provision-Befehl wurde nach allen Migrationen gegen eine ausschließlich dafür erzeugte, frische Testdatenbank ausgeführt. Geprüft: Einrichtung und Lizenzlaufzeit vorhanden, genau ein Kundenkonto mit Rolle admin, kein staff-/Superuserrecht, alle Ressourcen der fachlichen API ohne Datensätze. Der vorhandene lokale Datenbestand wurde dabei nicht verwendet.

Diese Prüfung bestätigt die leere Kundenanlage über Django. Sie bestätigt noch kein veröffentlichtes Container-Image, keine automatische VM-Bereitstellung, keine wiederaufnehmbare Installation und keinen Kundenzugriff über HTTPS. Docker ist auf dem Entwicklungsrechner installiert, sein Linux-Dienst lief beim Test nicht.

Planergänzung vom 13.10.2026. Registry-Paket, Einrichtungsassistent und Produktionsvorlagen sind hier als nächste Implementierung beschrieben. Es wurden keine Kundeninstanzen, Registry-Zugänge oder öffentlichen Domains angelegt.